

Web Technology

Subject Code: T2418102 | SEM-II Diploma (CSE)

Previous Year Question Paper 2024 (Even) — With Detailed Answers

GROUP A — MCQ / Fill in the Blanks (2 Marks Each × 10 = 20 Marks)

Q1(a). What is the correct syntax of doctype in HTML5?

Options: / <doctype html!> / <doctype html> / </doctype html>

■ Answer:

Explanation:

1. DOCTYPE declaration tells the browser which version of HTML is being used.
2. In HTML5, the correct and simplest syntax is:
3. It is written at the very top of every HTML file, before the <html> tag.
4. It is NOT a tag — it is a declaration. So it does not have a closing tag.
5. It is case-insensitive — and both work.

Example:

```
<!DOCTYPE html>
<html>
<head><title>My Page</title></head>
<body><p>Hello!</p></body>
</html>
```

Q1(b). Match the following: 1. HTML tag to make text italic 2. HTML tag to make text bold

Options: A. B.<bl> C.<italic> D.<i>

■ Answer: 1 → D (<i>) and 2 → A ()

Explanation:

1. To make text ITALIC in HTML, the correct tag is <i> ... </i>

Example: <i>Hello</i> → Hello (in italic)

2. To make text BOLD in HTML, the correct tag is ...

Example: Hello → Hello (in bold)

3. <bl> and <italic> are NOT valid HTML tags — they don't exist.
4. You can also use for italic (semantic) and for bold (semantic).

```
<b>This is bold text</b>
<i>This is italic text</i>
```

Q1(c). What is the correct HTML tag for largest heading and smallest heading?

Options: <h6><h2> / <h1><h5> / <h6><h1> / <h1><h6>

■ Answer: <h1> for largest, <h6> for smallest

Explanation:

1. HTML has 6 heading tags: <h1> to <h6>.
2. <h1> is the LARGEST heading — biggest font size, most important.
3. <h6> is the SMALLEST heading — smallest font size, least important.

4. As the number increases, the heading size DECREASES.
5. <h1> is usually used for the main title of the page.

```
<h1>This is the biggest heading</h1>
<h2>Second level heading</h2>
<h6>This is the smallest heading</h6>
```

Q1(d). A web page is a collection of websites. (True / False)

■ **Answer: FALSE**

Explanation:

1. This statement is WRONG — it is the opposite.
2. Correct statement: A WEBSITE is a collection of WEB PAGES.
3. A Web Page is a single document/file that can be viewed in a browser (e.g., one page of Facebook).
4. A Website is a collection of many related web pages under one domain (e.g., facebook.com is a website with millions of pages).
5. Think of it this way: Website = Book, Web Page = Page inside that book.

Q1(e). Which JavaScript method is used to write HTML output?

Options: Document.write() / document.output() / console.log() / document.writeHTML()

■ **Answer: document.write()**

Explanation:

1. document.write() is the JavaScript method used to write content directly into the HTML page.
2. It writes the output directly to the browser screen as part of the HTML document.
3. console.log() is used to print output in the browser's developer console — NOT on the webpage.
4. document.output() and document.writeHTML() do NOT exist in JavaScript.

```
<script>
document.write('Hello World!');
</script>
```

This will display 'Hello World!' directly on the webpage.

Q1(f). In client-server architecture, the component that processes the request and sends the response is?

Options: Client / Server / Protocol / Network

■ **Answer: Server**

Explanation:

1. In Client-Server architecture, there are two main components: Client and Server.
2. CLIENT: The user's device (browser/computer) that SENDS the request. Example: Your browser asking for google.com.
3. SERVER: The powerful computer that RECEIVES the request, PROCESSES it, and SENDS back the response (the web page).
4. Protocol: The set of rules used for communication (like HTTP, HTTPS) — it does NOT process the request.

5. So, the SERVER is the component that processes requests and sends responses.

Q1(g). Which keyword is used to define a variable in JavaScript?

Options: var / let / Both var and let / None of these

■ **Answer: Both var and let**

Explanation:

1. In JavaScript, variables can be declared using THREE keywords: var, let, and const.
2. var — Old way. Function-scoped. Can be re-declared and updated.

CODE:var name = 'Raval';

3. let — Modern way (ES6+). Block-scoped. Can be updated but NOT re-declared.

CODE:let age = 20;

4. const — For constants. Block-scoped. Cannot be updated or re-declared.

CODE:const PI = 3.14;

5. Since the question asks only about var and let, the answer is: Both var and let.
-

Q1(h). CSS Comments are placed within _____.

Options: // /* and */ / <* and *> /

■ **Answer: /* and */**

Explanation:

1. In CSS, comments are written between /* and */.
2. Anything written between /* and */ is ignored by the browser — it is only for the developer's reference.
3. // is used for single-line comments in JavaScript — NOT in CSS.
4. <* and *> and are not valid comment syntax in any language.

```
/* This is a CSS comment */
body {
background-color: blue; /* Sets background to blue */
}
```

Q1(i). How can you create an email link in HTML?

Options: <mail href='a@b'> / <mail>a@b</mail> / /

■ **Answer: **

Explanation:

1. To create an email link in HTML, we use the <a> anchor tag with href='mailto:emailaddress'.
2. When a user clicks this link, it opens their email application with the TO field already filled.
3. The keyword 'mailto:' is essential — without it, it will just be treated as a normal URL.
4. <mail> is NOT a valid HTML tag.

```
<a href='mailto:raval@gmail.com'>Send Email</a>
```

When user clicks 'Send Email', their email app opens with raval@gmail.com in the TO field.

Q1(j). What is the correct sequence of HTML tags for starting a webpage?

Options: Head,Title,HTML,Body / HTML,Body,Title,Head / Body,Title,HTML,Head / HTML,Head,Title,Body

■ Answer: HTML, Head, Title, Body

Explanation:

1. Every HTML page follows a specific structure/sequence.
2. First comes <html> — the root element that wraps everything.
3. Then comes <head> — contains meta information about the page.
4. Inside <head> comes <title> — defines the browser tab title.
5. Then comes <body> — contains all the visible content of the webpage.

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>
<p>Visible content here</p>
</body>
</html>
```

GROUP B — Short Answer Questions (4 Marks Each × 5 = 20 Marks)

Q2. Explain Heading Tags with example.

What are Heading Tags?

1. Heading tags in HTML are used to define headings or titles on a web page.
2. HTML provides 6 levels of headings: <h1> to <h6>.
3. <h1> is the BIGGEST (most important) and <h6> is the SMALLEST (least important).
4. Headings are displayed in bold by default and their font size decreases from h1 to h6.
5. Search engines use headings to understand the structure and content of a page.

Syntax:

```
<h1>This is Heading 1 - Biggest</h1>
<h2>This is Heading 2</h2>
<h3>This is Heading 3</h3>
<h4>This is Heading 4</h4>
<h5>This is Heading 5</h5>
<h6>This is Heading 6 - Smallest</h6>
```

Important Points:

6. Only one <h1> should be used per page (for the main title).
7. Headings should be used in order — don't skip from h1 to h4 directly.
8. Headings make the page structure clear for both users and search engines.

Q2 (OR). Explain JavaScript.

What is JavaScript?

1. JavaScript (JS) is a lightweight, interpreted programming language used to make web pages interactive and dynamic.
2. It runs directly in the web browser — no need to install anything extra.
3. JavaScript was created by Brendan Eich in 1995.
4. It is one of the three core technologies of the web: HTML (structure), CSS (style), JavaScript (behaviour).

What can JavaScript do?

5. Change HTML content dynamically — update text, images, or elements without reloading the page.
6. Validate user input in forms — check if the user filled in the correct data before sending.
7. Show/hide elements — create dropdown menus, pop-ups, and animations.
8. Communicate with servers — fetch data without refreshing the page (AJAX).
9. Create games, calculators, and interactive applications in the browser.

Simple Example:

```
<script>
document.write('Hello Raval!');
alert('Welcome to Web Technology!');
</script>
```

Key Features:

10. Case-sensitive — 'Name' and 'name' are different variables.
 11. Interpreted — Code runs line by line, no compilation needed.
 12. Object-based — Works with objects like document, window, etc.
-

Q3. Explain Ordered List with example.

What is an Ordered List?

1. An Ordered List is a list where each item is numbered in a specific order (1, 2, 3... or A, B, C...).
2. It is used when the sequence/order of items matters — like steps in a process.
3. In HTML, ordered list is created using the tag.
4. Each item inside the list is written using the (list item) tag.

Syntax:

```
<ol>
<li>First item</li>
<li>Second item</li>
<li>Third item</li>
</ol>
```

Output on screen:

1. First item
2. Second item
3. Third item

The 'type' Attribute — Different numbering styles:

5. type='1' — Default numbers: 1, 2, 3...
6. type='A' — Uppercase letters: A, B, C...
7. type='a' — Lowercase letters: a, b, c...
8. type='I' — Uppercase Roman numerals: I, II, III...
9. type='i' — Lowercase Roman numerals: i, ii, iii...

```
<ol type='A'>
<li>HTML</li>
<li>CSS</li>
<li>JavaScript</li>
</ol>
```

Output: A. HTML B. CSS C. JavaScript

Q3 (OR). List the types of Style Sheets.

What is CSS?

1. CSS (Cascading Style Sheets) is used to control the appearance (style) of HTML elements — colors, fonts, spacing, layout, etc.
2. There are THREE types of CSS based on where and how they are written.

Type 1 — Inline CSS:

3. Written directly inside an HTML element using the style attribute.
4. Applies style to ONLY that one specific element.
5. Highest priority among all three types.

```
<p style='color:red; font-size:18px;'>This is red text</p>
```

Disadvantage: Hard to manage — have to write style for each element separately.

Type 2 — Internal (Embedded) CSS:

6. Written inside the <style> tag in the <head> section of the HTML file.
7. Applies to the entire page but only to THAT single HTML file.

```
<head>
<style>
p { color: blue; font-size: 16px; }
</style>
</head>
```

Disadvantage: If you have 10 pages, you need to copy the style to all 10 files.

Type 3 — External CSS:

8. Written in a SEPARATE .css file and linked to the HTML page using the <link> tag.
9. Best practice — one CSS file can control the style of an entire website.

```
<!-- In HTML file -->
<head>
<link rel='stylesheet' href='style.css'>
</head>
/* In style.css file */
p { color: green; font-size: 14px; }
```

Advantage: Change one CSS file → entire website updates. Very efficient!

Q4. Write the format of an HTML Program.

Basic Format / Structure of HTML:

1. Every HTML file follows a standard structure/format.
2. This structure is like a skeleton — every HTML page must have these basic parts.

Complete HTML Format:

```
<!DOCTYPE html>
<html>
<head>
<title>My First Web Page</title>
</head>
<body>
<h1>Welcome to My Page</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

Explanation of each part:

3. — Tells the browser this is an HTML5 document. Written first, before everything.
4. <html> ... </html> — The ROOT element. All other tags are written inside this.
5. <head> ... </head> — Contains information ABOUT the page (not visible on screen). Includes title, CSS links, meta tags.
6. <title> ... </title> — The text that appears on the browser tab.
7. <body> ... </body> — Contains all VISIBLE content — text, images, links, tables, etc.

8. Tags come in pairs: opening tag <p> and closing tag </p>. Content goes between them.
 9. HTML is NOT case-sensitive — <P> and <p> are the same.
-

Q4 (OR). Describe the scope of variables in JavaScript.

What is Scope?

1. Scope means where a variable can be accessed (used) in the program.
2. If a variable is declared in one place, it may or may not be accessible in another place — that depends on its scope.
3. In JavaScript, there are THREE types of scope:

Type 1 — Global Scope:

4. A variable declared OUTSIDE all functions has global scope.
5. It can be accessed from ANYWHERE in the program — inside or outside functions.

```
var globalVar = 'I am global';  
function show() {  
  console.log(globalVar); // Works fine  
}  
show();  
console.log(globalVar); // Also works fine
```

Type 2 — Function (Local) Scope:

6. A variable declared INSIDE a function using var has function scope.
7. It can only be accessed inside that function — NOT outside.

```
function show() {  
  var localVar = 'I am local';  
  console.log(localVar); // Works fine  
}  
show();  
console.log(localVar); // ERROR! Cannot access outside function
```

Type 3 — Block Scope (let and const):

8. Variables declared with let or const inside { } braces have block scope.
9. They can only be accessed within that specific block (if, for loop, etc.).

```
if (true) {  
  let blockVar = 'I am block scoped';  
  console.log(blockVar); // Works fine  
}  
console.log(blockVar); // ERROR! Cannot access outside block
```

10. Key Difference: var ignores block scope — let and const respect block scope.
-

Q5. Explain Image tag with an example.

What is the Image Tag?

1. The tag is used to display images on a web page.
2. It is a self-closing (empty) tag — it does NOT have a closing tag.
3. The image is not inserted into the HTML page — it is linked from a file path or URL.

Syntax:


```
<img src='image.jpg' alt='Description of image'>
```

Important Attributes of tag:

4. src (Source) — Specifies the path or URL of the image. This is REQUIRED.

CODE: (local image)

CODE: (online image)

5. alt (Alternative Text) — Displays text if the image fails to load. Also used by screen readers for accessibility.

CODE:

6. width and height — Sets the size of the image in pixels.

CODE:

7. title — Shows a tooltip when user hovers the mouse over the image.

CODE:

Complete Example:

```
<!DOCTYPE html>
<html>
<body>
<h2>My Photo</h2>
<img src='photo.jpg' alt='My Photo' width='300' height='200'>
</body>
</html>
```

Q5 (OR). Explain the purpose of using HTML language.**What is HTML?**

1. HTML stands for HyperText Markup Language.
2. It is the standard language used to create and design web pages.
3. HTML uses TAGS (like <p>, <h1>,) to structure and format content on a webpage.
4. HTML is NOT a programming language — it is a markup language.

Purpose of HTML:

5. Creating Web Pages — HTML is used to build the basic structure of every web page on the internet. Every website you visit is built using HTML.
6. Structuring Content — HTML organizes content into headings, paragraphs, lists, tables, and sections, making it readable and structured.
7. Embedding Media — HTML allows you to add images, videos, audio, and animations to a web page using tags like , <video>, <audio>.
8. Creating Hyperlinks — HTML's <a> (anchor) tag connects web pages to each other through hyperlinks — this is what makes the 'web' a web.
9. Creating Forms — HTML forms (<form>, <input>, <button>) allow users to input data — used in login pages, registration, search bars, etc.
10. Platform Independent — HTML works on any browser (Chrome, Firefox, Edge) and any device (PC, mobile, tablet) — no special software needed.
11. Foundation for CSS and JavaScript — HTML provides the structure; CSS adds style; JavaScript adds behavior. All three work together on every modern website.

12. SEO (Search Engine Optimization) — Proper use of HTML heading tags, alt text, and meta tags helps search engines understand and rank web pages.

Q6. Differentiate between Dynamic and Static Website with example.

Static Website:

1. A Static website displays the SAME content to every user, every time they visit.
2. The content is fixed — it is written directly in HTML files and does not change unless the developer manually edits the code.
3. No database is used. No server-side processing happens.
4. Faster to load because pages are pre-built.
5. Example: A personal portfolio website, a company brochure website.

```
<!-- Static page – same content always -->  
<h1>Welcome to ABC Company</h1>  
<p>We are located in Mumbai.</p>
```

Dynamic Website:

6. A Dynamic website displays DIFFERENT content based on user, time, location, or interaction.
 7. Content is generated in real-time using server-side languages (PHP, Python) and databases.
 8. The page changes without the developer manually editing the code.
 9. Requires a database (like MySQL) and a server.
 10. Example: Facebook, Amazon, Gmail — content changes for every user.
- On Amazon: Different users see different recommended products based on their history.

Key Differences Table:

1. Content: Static = Fixed and same for all. Dynamic = Changes per user/request.
 2. Technology: Static = Only HTML/CSS. Dynamic = HTML + PHP/Python + Database.
 3. Speed: Static = Faster. Dynamic = Slightly slower (server processing needed).
 4. Cost: Static = Cheaper to host. Dynamic = More expensive.
 5. Maintenance: Static = Developer must manually update. Dynamic = Updates automatically.
 6. Interactivity: Static = Very limited. Dynamic = Highly interactive.
 7. Examples: Static = Portfolio sites. Dynamic = E-commerce, social media, email.
-

Q6 (OR). Explain Anchor Tag with an example.

What is the Anchor Tag?

1. The <a> tag (Anchor tag) is used to create HYPERLINKS in HTML.
2. A hyperlink allows the user to click and navigate to another page, website, file, or location.
3. The anchor tag is one of the most important tags in HTML — it is what makes the 'World Wide Web' connected.

Syntax:

```
<a href='URL'>Link Text</a>
```

The text between <a> and is what the user sees and clicks on.

Important Attributes:

4. href — Specifies the destination URL. This is REQUIRED for a link.
CODE:Go to Google
5. target — Specifies where to open the linked page.

_blank = Opens in a NEW tab

_self = Opens in the SAME tab (default)

CODE:Open Google in New Tab

6. title — Shows tooltip text when mouse hovers over the link.

CODE:Click Here

7. mailto — Creates an email link.

CODE:Email Us

8. Linking to a local page:

CODE:About Us

Complete Example:

```
<html>
<body>
<a href='https://www.google.com' target='_blank'>Visit Google</a>
<br>
<a href='mailto:test@gmail.com'>Send Email</a>
</body>
</html>
```

GROUP C — Long Answer Questions (6 Marks Each × 5 = 30 Marks)

Q7. Explain the structure of HTML with a small program.

What is HTML Structure?

1. Every HTML document follows a fixed structure — like a blueprint.
2. Without this structure, the browser may not display the page correctly.
3. The structure has 4 main parts: DOCTYPE, html, head, and body.

HTML Structure Diagram (Text):

→ Browser declaration

<html> → Root element (starts here)

<head> → Invisible info section

<title> → Browser tab title

</head>

<body> → Visible content section

content... → What user sees

</body>

</html> → Root element ends

Complete HTML Program:

```
<!DOCTYPE html>
<html>
<head>
<title>My First HTML Page</title>
</head>
<body>
<h1>Hello World!</h1>
<p>This is my first web page.</p>
<a href='https://google.com'>Click Here</a>
</body>
</html>
```

Explanation of each tag:

4. — Must be the FIRST line. Tells browser this is HTML5.
 5. <html> — Root element. Everything else goes inside this tag.
 6. <head> — Contains page information not visible to user.
>> <title> — Text shown on browser tab.
>> Can also include <link> (CSS), <meta> (SEO info), <script> (JS).
 7. <body> — Contains everything visible on the webpage.
>> <h1> — Main heading displayed on the page.
>> <p> — Paragraph of text.
>> <a> — Hyperlink.
 8. All tags must be properly OPENED and CLOSED.
 9. Tags can be NESTED — one tag inside another (indentation makes it readable).
-

Q7 (OR). Explain the need of Scripting Language in Web Technology.

What is a Scripting Language?

1. A Scripting Language is a programming language that automates tasks and adds dynamic behavior to web pages.
2. Unlike compiled languages (C, Java), scripting languages are interpreted — run line by line directly in the browser.
3. Most popular web scripting language: JavaScript (client-side), PHP (server-side).

Why Scripting Languages are needed in Web Technology:

4. Making Web Pages Interactive

Static HTML pages are boring — no response to user actions.

Scripting languages allow buttons to respond to clicks, forms to validate input, and content to change dynamically.

5. Form Validation

Before sending form data to the server, JavaScript can check if the user entered valid data (e.g., correct email format, no empty fields).

This saves server load and gives instant feedback to users.

6. Dynamic Content Updates

With AJAX (using JavaScript), parts of a page can be updated WITHOUT fully reloading the page.

Example: Facebook feed updates automatically without you refreshing the page.

7. Animation and Visual Effects

Scripting creates smooth animations, slideshows, dropdown menus, and pop-up windows.

8. Browser Control

JavaScript can control the browser — open new windows, handle cookies, detect browser type, and manage history.

9. Server-side Processing

Server-side scripting languages (like PHP) connect web pages to databases.

Example: When you log into Gmail, PHP/Python checks your username and password in the database.

10. Reduces Server Load

Client-side scripting (JavaScript) handles many tasks in the browser itself, reducing the number of requests sent to the server.

11. Building Web Applications

Modern web apps (Gmail, YouTube, Flipkart) are entirely built using scripting languages like JavaScript (React, Node.js).

Q8. Explain Cascading Style Sheets (CSS) with an example.

What is CSS?

1. CSS stands for Cascading Style Sheets.
2. CSS is used to control the VISUAL APPEARANCE of HTML elements — colors, fonts, sizes, spacing, layout, and positioning.
3. Without CSS, web pages would look plain with just black text on white background.
4. The word 'Cascading' means that styles are applied in a specific priority order — browser default → external → internal → inline.

Why CSS?

5. Separation of content (HTML) from design (CSS) — makes code cleaner and easier to manage.
6. One CSS file can style thousands of HTML pages — very efficient.
7. Easy to change the entire website design by editing just one CSS file.

CSS Syntax:

```
selector {  
  property: value;  
}
```

Selector: Which HTML element to style (e.g., p, h1, .class, #id)

Property: What to change (e.g., color, font-size, background)

Value: What value to set (e.g., red, 16px, blue)

Types of CSS (Brief):

8. Inline CSS — style attribute directly in HTML tag.
9. Internal CSS — <style> tag in <head> section.
10. External CSS — separate .css file linked with <link> tag.

Complete Example:

```
<!DOCTYPE html>  
<html>  
  <head>  
    <title>CSS Example</title>  
    <style>  
      body {  
        background-color: lightblue;  
        font-family: Arial;  
      }  
      h1 {  
        color: darkblue;  
        text-align: center;  
      }  
      p {  
        color: black;  
        font-size: 16px;  
      }  
    </style>  
  </head>  
  <body>  
    <h1>Welcome to My Page</h1>  
    <p>This paragraph is styled with CSS.</p>  
  </body>  
</html>
```

Result: Light blue background, dark blue centered heading, black paragraph text.

Q8 (OR). Explain Architecture of WWW in detail.

What is WWW?

1. WWW stands for World Wide Web.
2. It is a system of interconnected documents (web pages) and other resources, linked by hyperlinks, and accessible via the internet.
3. WWW was invented by Tim Berners-Lee in 1989.
4. WWW is NOT the internet — the internet is the physical network; WWW is a service that runs ON the internet.

Architecture of WWW — Three Main Components:

5. Component 1 — Client (Web Browser)

The user's device and browser (Chrome, Firefox, Safari).

The client SENDS requests to the server.

It receives the response (HTML, CSS, JS files) and RENDERS it into a visible web page.

Examples: Google Chrome, Mozilla Firefox, Microsoft Edge.

6. Component 2 — Web Server

A powerful computer that STORES web pages and resources.

It RECEIVES requests from clients and SENDS back the appropriate response.

Examples: Apache, Nginx, IIS (Internet Information Services).

7. Component 3 — HTTP/HTTPS Protocol

HTTP (HyperText Transfer Protocol) is the set of RULES for communication between client and server.

HTTPS is the secure version (encrypted communication).

Every web request and response follows HTTP protocol.

How WWW Works — Step by Step:

8. Step 1 — User types a URL (e.g., www.google.com) in the browser.
9. Step 2 — Browser sends an HTTP request to the DNS server to find the IP address of google.com.
10. Step 3 — DNS returns the IP address of Google's server.
11. Step 4 — Browser sends an HTTP request to Google's web server.
12. Step 5 — Google's web server processes the request and sends back the HTML/CSS/JS files.
13. Step 6 — Browser receives the files and renders the Google homepage on your screen.

Other Key Components:

14. URL (Uniform Resource Locator) — The address of a web page. Example:

<https://www.google.com>

15. HTML — Language used to create web pages.

16. Hyperlinks — Connect one page to another — making the web a 'web'.

17. DNS (Domain Name System) — Converts domain names to IP addresses.

Q9. Describe rowspan, colspan, cellpadding and cellspacing in HTML with example.

HTML Tables — Basic Structure:

1. Tables in HTML are created using <table>, <tr> (table row), <th> (table header), and <td> (table data) tags.

1. rowspan:

2. rowspan allows a table cell to SPAN (extend) across multiple ROWS.

3. rowspan='2' means the cell will occupy 2 rows instead of 1.

```

<table border='1'>
<tr>
<td rowspan='2'>Merged Row</td>
<td>Row 1, Col 2</td>
</tr>
<tr>
<td>Row 2, Col 2</td>
</tr>
</table>

```

Result: 'Merged Row' cell stretches across 2 rows.

2. colspan:

4. colspan allows a table cell to SPAN across multiple COLUMNS.
5. colspan='2' means the cell will occupy 2 columns instead of 1.

```

<table border='1'>
<tr>
<td colspan='2'>Merged Column Cell</td>
</tr>
<tr>
<td>Col 1</td>
<td>Col 2</td>
</tr>
</table>

```

Result: 'Merged Column Cell' stretches across 2 columns.

3. cellpadding:

6. cellpadding defines the SPACE between the cell content and the cell border.
7. It is like the inner margin of a cell.

```

<table border='1' cellpadding='10'>
<tr><td>Data 1</td><td>Data 2</td></tr>
</table>

```

Result: 10px of space between text and cell border.

4. cellspacing:

8. cellspacing defines the SPACE between individual cells (the gap between cells).

```

<table border='1' cellspacing='10'>
<tr><td>Data 1</td><td>Data 2</td></tr>
</table>

```

Result: 10px gap between each cell.

Quick Memory Trick:

- cellPADDING = padding INSIDE the cell (content to border)
- cellSPACING = spacing BETWEEN cells (cell to cell)

Q9 (OR). Explain Control Flow Statements in JavaScript with example.

What are Control Flow Statements?

1. Control flow statements control the ORDER in which code is executed.

2. Without control flow, code runs top to bottom, line by line.
3. Control flow statements allow the program to make decisions, repeat actions, and jump to different parts.

Type 1 — if / else if / else (Decision Making):

4. Used to execute different code based on a condition.

```
var marks = 75;
if (marks >= 90) {
  console.log('Grade A');
} else if (marks >= 60) {
  console.log('Grade B'); // This runs
} else {
  console.log('Grade C');
}
```

Type 2 — switch (Multiple Choices):

5. Used when you have many possible values for one variable.

```
var day = 'Monday';
switch(day) {
  case 'Monday': console.log('Start of week'); break;
  case 'Friday': console.log('End of week'); break;
  default: console.log('Midweek');
}
```

Type 3 — for loop (Repeat fixed times):

6. Repeats a block of code a specific number of times.

```
for (var i = 1; i <= 5; i++) {
  console.log('Number: ' + i);
}
```

Output: Number: 1, Number: 2 ... Number: 5

Type 4 — while loop (Repeat while condition is true):

7. Repeats as long as a condition remains true.

```
var i = 1;
while (i <= 3) {
  console.log('Count: ' + i);
  i++;
}
```

Type 5 — do...while loop:

8. Like while loop but runs the code at least ONCE before checking the condition.

```
var i = 1;
do {
  console.log('Value: ' + i);
  i++;
} while (i <= 3);
```

Q10. Explain Web Server. List the features of Web Server.

What is a Web Server?

1. A Web Server is a computer (hardware) or software program that stores, processes, and delivers web pages to users when they request them.
2. When you type a URL in your browser, your browser sends a request to the web server. The web server finds the page and sends it back to your browser.
3. Web servers use the HTTP or HTTPS protocol for communication.
4. Popular Web Servers: Apache HTTP Server, Nginx, Microsoft IIS, LiteSpeed.

How a Web Server works:

5. Step 1 — User types a URL in the browser (e.g., www.example.com).
6. Step 2 — Browser sends an HTTP request to the web server.
7. Step 3 — Web server locates the requested file (HTML page, image, etc.).
8. Step 4 — Web server sends the file back as an HTTP response.
9. Step 5 — Browser receives the file and displays the web page.

Features of a Web Server:

10. HTTP/HTTPS Support — Handles communication using HTTP and HTTPS protocols. HTTPS encrypts data for secure communication.
11. Static and Dynamic Content Serving — Can serve static files (HTML, images) and dynamic content (generated by PHP, Python).
12. Multiple Requests Handling — Can handle thousands of simultaneous requests from different users at the same time.
13. Security — Supports SSL/TLS certificates for HTTPS. Can block unauthorized access and protect against attacks.
14. Virtual Hosting — One physical server can host multiple websites with different domain names.
15. Logging and Monitoring — Records all requests and errors in log files for monitoring and debugging.
16. Load Balancing — Distributes incoming traffic across multiple servers to prevent overload.
17. Compression — Compresses files before sending to reduce data transfer time and improve speed.
18. Caching — Stores frequently requested content in memory to serve it faster next time.
19. CGI Support — Allows server-side scripts (PHP, Python) to run on the server to generate dynamic pages.

Q10 (OR). Explain Exception Handling in JavaScript with a program.

What is Exception Handling?

1. An Exception is an unexpected error that occurs during program execution.
2. If an error is not handled, the program STOPS running and shows an error.
3. Exception Handling allows the program to CATCH errors, handle them gracefully, and CONTINUE running instead of crashing.
4. JavaScript uses try, catch, finally, and throw for exception handling.

Keywords:

5. try — Code that might throw an error is written here.
6. catch — Code that runs when an error occurs (to handle the error).
7. finally — Code that ALWAYS runs, whether there is an error or not.
8. throw — Manually creates a custom error.

Syntax:

```
try {  
    // Code that might cause an error  
} catch (error) {  
    // Code to handle the error  
} finally {  
    // Code that always runs  
}
```

Complete Example Program:

```
<!DOCTYPE html>  
<html>  
<body>  
<script>  
try {  
    var x = 10;  
    var y = 0;  
    if (y === 0) {  
        throw 'Cannot divide by zero!';  
    }  
    var result = x / y;  
    document.write('Result: ' + result);  
}  
catch (error) {  
    document.write('Error caught: ' + error);  
}  
finally {  
    document.write('<br>Program continues normally.');}  
</script>  
</body>  
</html>
```

Output on screen:

Error caught: Cannot divide by zero!

Program continues normally.

Why use Exception Handling?

9. Prevents the program from crashing on errors.
10. Gives meaningful error messages to users instead of confusing technical errors.
11. Allows the program to continue running after handling the error.

Q11. Explain Table tag and their attributes with an example.**What is a Table in HTML?**

1. An HTML Table is used to display data in rows and columns — like a spreadsheet.
2. Tables are created using a set of related HTML tags.

Main Table Tags:

3. <table> — Creates the table. All other table tags go inside this.
4. <tr> (table row) — Creates a horizontal row inside the table.
5. <th> (table header) — Creates a header cell. Text is BOLD and CENTERED by default.
6. <td> (table data) — Creates a normal data cell.
7. <caption> — Adds a title/caption above the table.

Important Table Attributes:

8. border — Adds border around cells. border='1' shows borders.
9. cellpadding — Space between cell content and cell border.
10. cellspacing — Space between cells.
11. width — Sets the width of the table.
12. align — Aligns the table on the page (left, center, right).
13. bgcolor — Sets background color of table or cell.
14. colspan — Makes a cell span multiple columns.
15. rowspan — Makes a cell span multiple rows.

Complete Example:

```
<!DOCTYPE html>
<html>
<body>
<table border='1' cellpadding='8' cellspacing='2' width='400'>
<caption>Student Results</caption>
<tr>
<th>Roll No</th>
<th>Name</th>
<th>Marks</th>
</tr>
<tr>
<td>1</td>
<td>Raval</td>
<td>85</td>
</tr>
<tr>
<td>2</td>
<td>Amit</td>
<td>90</td>
</tr>
</table>
</body>
</html>
```

This creates a table with a caption, header row, and 2 data rows with border and padding.

Q11 (OR). Explain JavaScript and the features of JavaScript.

What is JavaScript?

1. JavaScript is a high-level, interpreted, object-based scripting language.

2. It is used to make web pages interactive and dynamic.
3. JavaScript runs in the browser — no installation needed on the user's computer.
4. Created by Brendan Eich in 1995. Standardized as ECMAScript.
5. Along with HTML and CSS, JavaScript is one of the three core technologies of the web.

```
<script>
alert('Hello! I am JavaScript!');
</script>
```

Features of JavaScript:

6. Interpreted Language — JavaScript code is executed line by line by the browser. No compilation step is needed.
7. Client-Side Execution — JavaScript runs on the USER's browser (client-side), reducing the load on the server.
8. Platform Independent — JavaScript works on all browsers (Chrome, Firefox, Edge) and all operating systems (Windows, Mac, Linux).
9. Object-Based — JavaScript uses objects like document, window, Math, Array, etc. to perform operations.
10. Event-Driven — JavaScript responds to user actions (events) like clicking a button, moving the mouse, or pressing a key.

```
document.getElementById('btn').onclick = function() {
alert('Button Clicked!');
}
```

11. Dynamic Typing — You don't need to declare the data type of a variable. JavaScript figures it out automatically.

```
var x = 10; // number
var y = 'Hello'; // string – no type declaration needed
```

12. Case-Sensitive — JavaScript is case-sensitive. 'Name' and 'name' are two different variables.
 13. Functions as First-Class Objects — Functions can be stored in variables, passed as arguments, and returned from other functions.
 14. Asynchronous Programming — JavaScript supports async operations (AJAX, Promises, async/await) — allows fetching data without reloading the page.
 15. Rich Library Support — JavaScript has powerful libraries and frameworks: React, Angular, Vue (frontend), Node.js (backend).
 16. Form Validation — JavaScript validates user input on the client side before sending data to the server.
-

QUICK REVISION — GROUP A ANSWER KEY

Q.No	Question (Short)	Answer
Q1(a)	Correct DOCTYPE syntax in HTML5	<!DOCTYPE html>
Q1(b)	Tag for italic / Tag for bold	<i> /
Q1(c)	Largest heading / Smallest heading	<h1> / <h6>
Q1(d)	Web page = collection of websites	FALSE (Website = collection of pages)
Q1(e)	JS method to write HTML output	document.write()
Q1(f)	Processes request in client-server	Server
Q1(g)	Keyword to define variable in JS	Both var and let
Q1(h)	CSS comments placed within	/* and */
Q1(i)	How to create email link in HTML	
Q1(j)	Correct sequence of HTML tags	HTML, Head, Title, Body

All the best Raval! Web Tech bhi crack karega! ■■